

Open in app ↗

Sign up

Sign in

Medium

 Search Write

Towards Dev

Exploring Hugging Face: Sentiment Analysis

Sentiment Analysis with Transformer Models



Okan Yenigün

5 min read · Dec 8, 2023



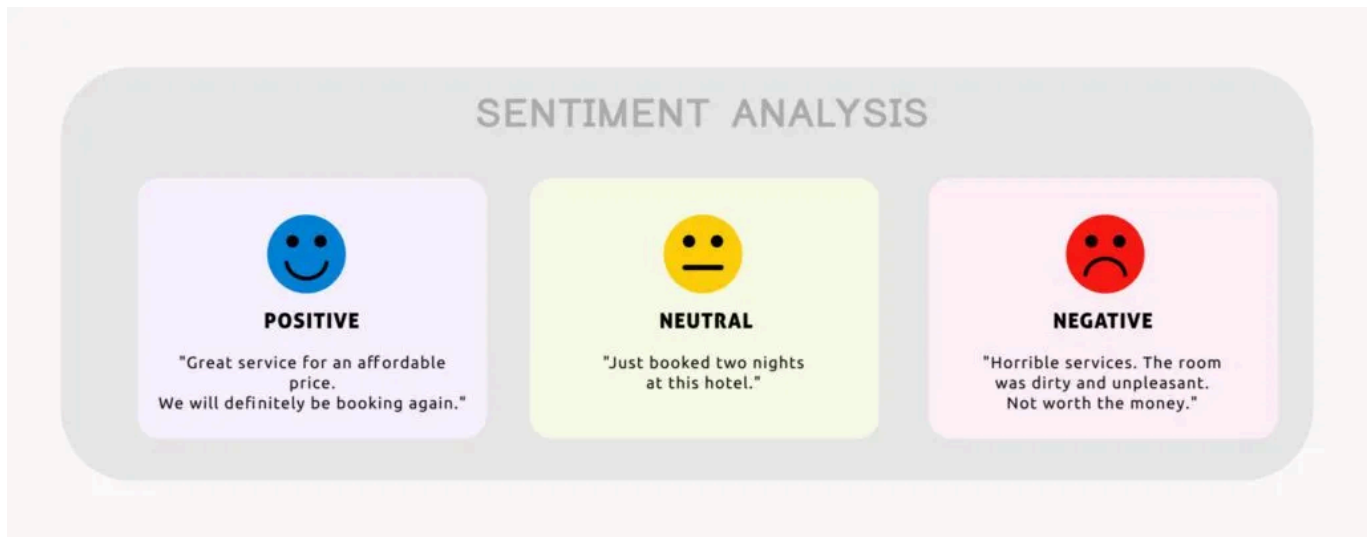
--





Photo by [Brett Jordan](#) on [Unsplash](#)

Sentiment Analysis in NLP is a task where the aim is to identify and categorize opinions or sentiments expressed in texts. This process involves determining whether the sentiment is positive, negative, or neutral, and sometimes even more fine-grained like happy, sad, angry, etc., can be identified.



Sentiment analysis. [Source](#)

Sentiment analysis is widely used in business and social media monitoring, where it helps in understanding customer opinions, market trends, and public sentiments toward certain topics.

Hugging Face offers a wide array of pre-trained models, but **selecting the right one** for a specific sentiment analysis task is crucial. Each model has its strengths and weaknesses, and some may be more suited for certain types of text or languages.

Additionally, fine-tuning these models with domain-specific data can significantly improve performance but requires additional data and computational resources.

[mrm8488/distilroberta-finetuned-financial-news-sentiment-analysis](#) is a fine-tuned financial model.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification, pipe

tokenizer = AutoTokenizer.from_pretrained("mrm8488/distilroberta-finetuned-finan
model = AutoModelForSequenceClassification.from_pretrained("mrm8488/distilrobert
```

```
nlp = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)

text = """XYZ Corporation's stock soared by 20% after reporting
record-breaking annual profits and announcing a significant dividend
increase for shareholders.
"""

sentiment = nlp(text)
print(sentiment)

# [{'label': 'positive', 'score': 0.9996914863586426}]

text = """Yesterday, the local botanical garden unveiled a rare
collection of orchids, attracting a large number of nature enthusiasts
and photographers excited to witness the unique floral display.
"""

sentiment = nlp(text)
print(sentiment)

# [{'label': 'neutral', 'score': 0.6923548579216003}]
```

The model worked well for a financial text, but for the text which is completely unrelated to finance, it struggled and could not decide about the sentiment.

Sentiment analysis models can struggle with understanding context and detecting **sarcasm or irony**. This is because these elements often rely on subtle cues and background knowledge, which can be challenging for a model to grasp. Misinterpreting such nuances can lead to incorrect sentiment classification.



Bazinga. [Source](#)

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForSequenceClassification, pipeline

tokenizer = AutoTokenizer.from_pretrained("nlptown/bert-base-multilingual-uncased")
model = AutoModelForSequenceClassification.from_pretrained("nlptown/bert-base-multilingual-uncased")

nlp = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)

text = "Wow, another superhero movie bombed at the box office - so surprising!"

sentiment = nlp(text)
print(sentiment)

# [{'label': '5 stars', 'score': 0.9021768569946289}]
```

Pre-trained models can inherit **biases** present in their training data. This can lead to skewed results, especially when analyzing texts related to gender,

race, or other sensitive topics. It's important to be aware of these biases and take steps to mitigate them.

Sentiment is often expressed differently across languages and cultures. Models trained predominantly on data from one language or cultural context may not perform well on data from another. This is a significant challenge in creating universally effective sentiment analysis models. There are some models trained specifically on some languages and some “**multilingual**” models that can answer more than one language.

nlptown/bert-base-multilingual-uncased-sentiment. [Source](#)

yeniguno/democracy-sentiment-analysis-turkish-roberta · Hugging Face

We're on a journey to advance and democratize artificial intelligence through open source and open science.

huggingface.co

```
from transformers import pipeline

# Load the model from Hugging Face
sentiment_model = pipeline(model="yeniguno/democracy-sentiment-analysis-turkish-

# Example text input
response = sentiment_model("En iyisi devletin tüm gücünü tek bir lidere verelim")

# Print the result
print(response)
# [{'label': 'negative', 'score': 0.9617443084716797}]

# Example text input
response = sentiment_model("Birçok farklı sesin çıkması zaman alıcı ve karmaşık

# Print the result
print(response)
# [{'label': 'positive', 'score': 0.958978533744812}]

# Example text input
response = sentiment_model("Bugün hava yağmurlu.")

# Print the result
print(response)
# [{'label': 'neutral', 'score': 0.9915837049484253}]
```

Texts often contain ambiguous or **mixed** sentiments, which can be challenging for models to interpret accurately. For instance, a review might contain both positive and negative aspects, and determining the overall sentiment can be complex.

Get Okan Yenigün's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

The **quality of input data** significantly affects model performance. Texts with a lot of noise, such as misspellings, slang, or poor grammar, can be challenging to process and understand. Effective preprocessing is necessary to clean and standardize the data.

Analysis outputs can be in various ways:

- Binary output

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification, pipe

tokenizer = AutoTokenizer.from_pretrained("siebert/sentiment-roberta-large-english")
model = AutoModelForSequenceClassification.from_pretrained("siebert/sentiment-roberta-large-english")

nlp = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)

text_good = "This product is good."
text_bad = "This product is bad."
text_neutral = "This product is not good, neither bad."

for text in [text_good, text_bad, text_neutral]:
    sentiment = nlp(text)
    print(sentiment)

"""
[{'label': 'POSITIVE', 'score': 0.9988435506820679}]
[{'label': 'NEGATIVE', 'score': 0.9995039701461792}]
[{'label': 'NEGATIVE', 'score': 0.999470055103302}]
"""
```

- Three category

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification, pipe
```

```
tokenizer = AutoTokenizer.from_pretrained("lxyuan/distilbert-base-multilingual-c
model = AutoModelForSequenceClassification.from_pretrained("lxyuan/distilbert-ba

nlp = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)

text_good = "This product is good."
text_bad = "This product is bad."
text_neutral = "This is just a neutral question"

for text in [text_good, text_bad, text_neutral]:
    sentiment = nlp(text)
    print(sentiment)

"""
[{'label': 'positive', 'score': 0.9819211363792419}]
[{'label': 'negative', 'score': 0.9518771767616272}]
[{'label': 'neutral', 'score': 0.5191274881362915}]
"""
```

5 star

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification, pipe

tokenizer = AutoTokenizer.from_pretrained("nlptown/bert-base-multilingual-uncase
model = AutoModelForSequenceClassification.from_pretrained("nlptown/bert-base-mu

nlp = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)

text_good = "This product is good."
text_bad = "This product is bad."
text_neutral = "This is just a neutral question"

for text in [text_good, text_bad, text_neutral]:
    sentiment = nlp(text)
    print(sentiment)

"""
[{'label': '4 stars', 'score': 0.4784603416919708}]
[{'label': '1 star', 'score': 0.7462035417556763}]
```

```
[{'label': '1 star', 'score': 0.3701822757720947}]  
""
```

You can find other models that give different types of outputs out there in Hugging Face.

Next:

Exploring Hugging Face: ABSA

Aspect-Based Sentiment Analysis

medium.com

Fine Tuning:

Fine-Tuning Sentiment Analysis Model

How to Fine-Tune Sentiment Analysis Models with Hugging Face and PyTorch

medium.com

Read More

Exploring Hugging Face: An Introduction

The First Chapter in Our AI Exploration Series

medium.com

The Python Programmer's Toolkit: Essential Libraries for Translation

Translation Libraries in Python

medium.com

A Nostalgic Look Back at GPT-1

The Humble Beginnings of a Revolution

pub.aimind.so

Introduction to Elastic Search

Essentials of ELK Stack

blog.devgenius.io

Spearman's Rank Correlation

Simplified Spearman's Rank Correlation in Python

towardsdev.com

Artificial Intelligence

Transformers

NLP

Sentiment Analysis

Machine Learning



Published in Towards Dev

9.91K followers · Last published 6 hours ago

A publication for sharing projects, ideas, codes, and new theories.



Written by Okan Yenigün

3.4K followers · 418 following

AI, Data & Software

No responses yet



